Job In Tech Program – Final Exam Project Specification

Institution : Ynov Campus Maroc
Program : Job In Tech
Subject : DevOps
Date : 08 / 01 / 2026
Duration : 15 days
Supervisor : LOUKACH EL-Mehdi

MAROC ynov CAMPUS

Jobintech

Project Overview

You are working in a start-up company to deliver a Minimum Viable Product representing the solution and the idea to solve the main problem of your company's mission.

The project consists of building and deploying a full-stack web application inspired by cloud-based platforms. The core concept revolves around the development of a web application, testing functionalities, and real-time deployment. The platform must be designed as a scalable, secure, and maintainable web application, suitable for a production environment.

The application will be composed of a RESTful back-end service developed with Node.js or GoLang, a modern front-end single-page application built with React.js, and a robust database layer using either SQL or NoSQL technologies. The system must follow clean architectural principles, separation of concerns, and professional development workflows.

By the end of the 15 days, each team must demonstrate the ability to deliver software the DevOps way: fast feedback, repeatability, automation, and safe operations.

Your solution must cover the main learning blocks from the training program (systems and networking, Go programming, databases, Git and Agile, CI/CD with GitLab and/or Jenkins, Docker, Kubernetes, Terraform, Ansible, ArgoCD, Prometheus/Grafana, Jaeger, ELK/EFK and DevSecOps practices).

Evaluation focuses on integration rather than isolated tools. A perfect project is not the one with the most tools, but the one where each component has a clear purpose and a clean interface with the rest of the system.

Reviewers will look for: traceability from a Git commit to a running deployment, reproducible infrastructure, automated tests, measurable service health, and security controls applied by default.

This document describes the end-to-end capstone project your team must deliver. The goal is to build a production-like software delivery system and a cloud-native application, connecting development, version control, CI/CD, infrastructure, containers, Kubernetes, GitOps, observability and security into one coherent platform. Deliverable: one working platform per team, deployed and observable, demonstrated live.

Scenario and application scope :

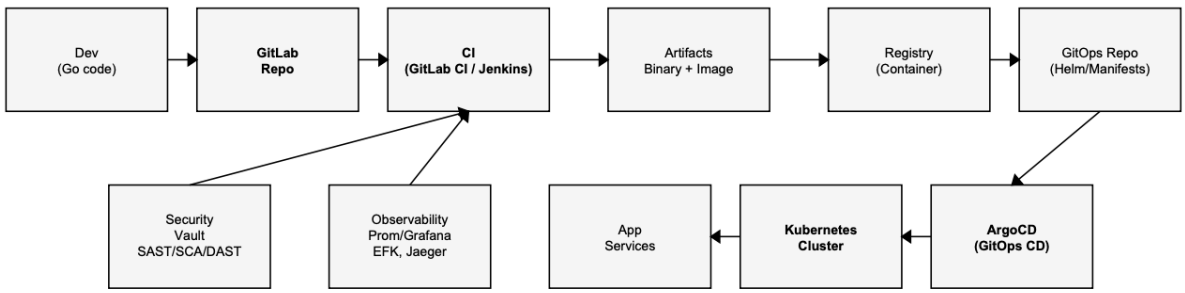
You will build a small but realistic cloud-native platform: a set of Go microservices that expose REST APIs, persist data in databases, and run on Kubernetes.
The project is intentionally designed to make every DevOps component necessary: source control, CI, image building, registry, deployment, observability, and security.
Recommended scenario (feel free to rename): “Campus Market” - a simple marketplace where users browse products and place orders.
Core services (minimum):
- API Gateway / BFF: single external entry point, routes requests to internal services.
- Auth Service: user registration/login, issues JWT tokens (PostgreSQL).
- Catalog Service: product catalog with search (MongoDB).
- Order Service: create and list orders (PostgreSQL).
- Redis: cache product search results or sessions.
Non-functional requirements: versioned APIs, structured logging, Prometheus metrics endpoint, OpenTelemetry tracing, health checks, and graceful shutdown.

Trainees are not required to develop a brand new API, programs developed during the training or a chosen project but the trainee will be acceptable to use for deployment if it meets the minimum requirement.

Reference architecture and component relationships :

This capstone is not “a Kubernetes exercise”. It is a system of systems. The most important skill is understanding how responsibilities move across layers: developer tools (Go, Git), delivery automation (CI/CD), runtime platform (Kubernetes), and operational feedback (monitoring, logging, tracing, security alerts).

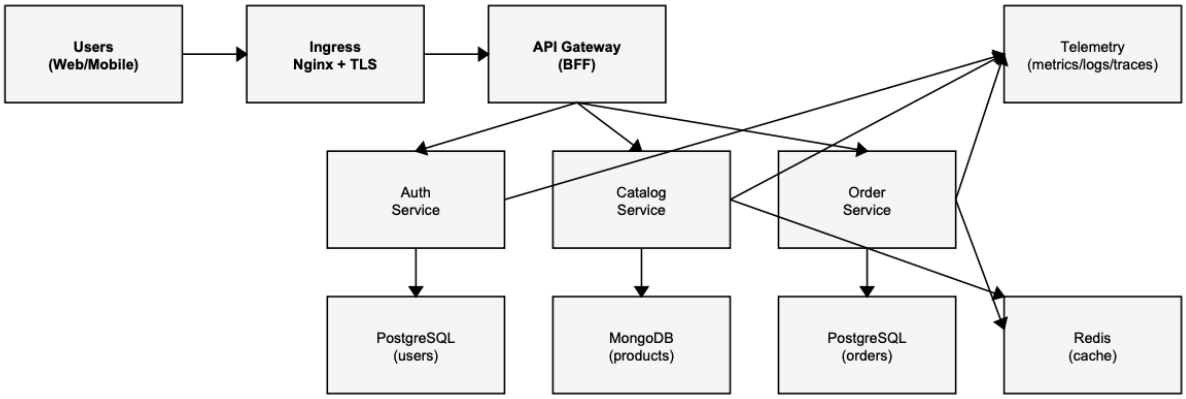
End-to-end delivery flow :



The diagram below shows the expected relationships from commit to production and back to engineering feedback.

Key idea: the flow is not one-directional. Observability and security are feedback loops that continuously improve code and infrastructure. Metrics, traces, logs and vulnerabilities should create tickets, influence priorities, and block risky releases.

Runtime architecture :



At runtime, the application is a set of independent services deployed as containers, connected by stable contracts (APIs) and backed by stateful components.
Where the relationships matter: Kubernetes gives you scheduling, service discovery and scaling, but your services must be designed for it: stateless processes, externalized configuration, idempotent DB migrations, readiness probes, and predictable logs/metrics. The platform can only be as reliable as the weakest contract between components.

Development and versioning workflow (Git) :

Your Git repository is the single source of truth for application code. Git history is also the backbone of traceability: every deployed version must correspond to an immutable commit SHA and a semantic version tag (for example v1.3.0).
Minimum requirements: -
One main repository for application code (mono-repo is allowed) and one repository for GitOps deployment definitions (Helm charts / Kustomize manifests).
- A clear branching strategy (for example: trunk-based with short-lived feature branches, or GitFlow if you can justify it).
- Merge requests with code review and mandatory pipeline success before merge.
- Semantic versioning and changelog generation (manual or automated).
- Git tags/releases drive deployable versions.
The core relationship to understand here is: version control governs deployment. CI produces artifacts from commits; CD deploys artifacts referenced by versions; observability tells you if that

version is healthy. If you cannot answer “what code is currently running?”, your system is not operationally safe.

CI pipeline (build, test, quality, security) :

CI is where engineering discipline becomes automated. A commit must trigger an automated pipeline that provides fast feedback: compilation, unit tests, linting, quality checks, and security scans. CI also produces immutable artifacts that CD can deploy.
Minimum CI stages:
1) Validate: formatting, linting, dependency checks.
2) Test: unit tests + coverage; integration tests with ephemeral DB containers if possible.
3) Build: build Go binaries; build Docker images with reproducible Dockerfiles.
4) Security: SAST + SCA + container image scanning; fail the pipeline on critical issues.
5) Publish: push Docker images to a registry; publish a Helm chart package or update a GitOps repo.

You may implement CI with GitLab CI (recommended) and optionally demonstrate Jenkins by running a second pipeline for nightly regression/performance tests. What matters is not the tool name but the ability to explain why each gate exists and how it protects production.

Example conceptual pipeline (pseudo-code)

```
stages: [validate, test, build, security, publish]
validate: run gofmt + golangci-lint
test: run go test ./... + coverage
build: docker build -t registry/app:$CI_COMMIT_SHA .
security: run SAST + dependency scan + image scan
publish: push image; bump chart version; open MR to GitOps repo
```

Containerization and local developer experience :

Containers are the contract between development and operations. A good container image is portable, minimal, secure, and configured from the outside. Your Dockerfiles should use multi-stage builds, non-root users, pinned base images, and explicit exposed ports.

Minimum requirements:
- A docker-compose file for local development that starts services + databases quickly.
- Images are built by CI, not manually on laptops, and are stored in a registry.
- Configuration is injected via environment variables and ConfigMaps/Secrets, not hard-coded.
The relationship to highlight: the same image goes through every environment. You do not “rebuild for production”. Instead you promote the same tested artifact from dev to test to prod, changing only configuration and infrastructure.

Kubernetes deployment design :

Kubernetes is the execution environment. It provides scheduling, self-healing and scaling, but you must model your application correctly using the right objects: Deployments for stateless services, StatefulSets for databases if self-hosted, Services for discovery, Ingress for external access, and proper Namespaces to isolate environments.

Minimum Kubernetes requirements:
- Separate namespaces for at least dev and prod (or dev/staging/prod).
- Ingress with TLS (self-signed is acceptable in the lab).
- Readiness and liveness probes; resource requests/limits.
- ConfigMaps for non-secret configuration; Secrets for credentials (preferably sourced from Vault).
- Horizontal Pod Autoscaler (HPA) for at least one service based on CPU or custom metrics.
- Persistent volumes for stateful components (DBs), including a backup/restore mechanism.

The relationship to emphasize: Kubernetes turns “deploy” into a desired state. Your delivery system (GitOps) commits a desired state; Kubernetes reconciles it. This is why deployment definitions must be versioned, reviewable, and reproducible.

Infrastructure as Code (Terraform) and Configuration Management (Ansible) :

Infrastructure is part of the product. In this capstone, your team must define infrastructure in code and make it reproducible. Terraform is used to provision resources (networks, VMs, firewalls, load balancers, managed services). Ansible is used to configure machines and install software in an idempotent way.

Minimum requirements:
- Terraform project structured with modules and environments (workspaces or separate state).
- Remote state management (for example object storage) and state locking if available.
- At least one real provisioning target: cloud VMs, a Kubernetes cluster node pool, object storage, or managed database (depending on your lab).
- Ansible roles/playbooks to install: container runtime, Kubernetes prerequisites, monitoring agents, and baseline security hardening.

The relationship to understand: CI/CD depends on infrastructure stability. If infrastructure is manual, deployments are fragile and slow. If infrastructure is code, environments become

disposable and consistent, enabling reliable testing, repeatable incidents drills, and faster onboarding.

GitOps delivery with ArgoCD :

GitOps is the heart of the “relationship-first” approach. Your GitOps repository describes the desired state of each environment: which image version, which Helm values, which Kubernetes objects. ArgoCD continuously compares that desired state with what is actually running and reconciles differences.

Minimum requirements:
- One ArgoCD application per environment (or per service).
- Automated sync enabled for dev; controlled sync (manual approval) for prod is encouraged.
- Clear promotion mechanism: changing a Git reference promotes a version (for example, updating a Helm values file from image tag v1.2.0 to v1.3.0).
- Rollback procedure demonstrated (revert commit or switch back tag).

The relationship to emphasize: Git becomes the deployment API. Instead of clicking a UI to deploy, you change Git, review it, merge it, and ArgoCD applies it. This makes every change auditable, reproducible, and easy to roll back.

Observability: monitoring, logging, tracing :

A system you cannot observe is a system you cannot operate. Observability is not “install Prometheus”. It is the discipline of making the internal state visible through three signals: metrics, logs and traces.

Minimum requirements:
- Prometheus scrapes application and cluster metrics;
Grafana dashboards show service health (latency, error rate, throughput) and infrastructure health (CPU, memory, disk).
- Alertmanager triggers at least 3 meaningful alerts (for example: high 5xx rate, pod crash loop, database down).
- EFK/ELK collects logs from Kubernetes; logs are structured (JSON) and searchable by correlation IDs.
- Jaeger (or OpenTelemetry + Jaeger) traces a request from gateway to downstream services and databases. The relationship to show in your demo: a production incident should be diagnosable. Start from an alert, open the dashboard, follow a trace to identify the slow component, then use logs to confirm the root cause. If these tools are deployed but not connected to the way your app behaves, you will not get credit.

DevSecOps: secrets, scanning, hardening :

Security must be built into the delivery system. In this project, you will apply “shift-left” security (catch issues before production) and “secure-by-default” runtime controls.

Minimum requirements:
- Secret management: use Vault (or an equivalent secrets manager) to store sensitive values. - Pipelines must not print secrets.
- Kubernetes Secrets should be sourced from Vault or injected securely.
- Pre-commit hooks: prevent committing secrets and enforce formatting/linting.
- SAST + SCA: static and dependency scanning integrated into CI with a fail policy for high/critical findings.
- Container security: Dockerfile best practices, image scanning, and least-privilege runtime (non-root, read-only FS if possible).
- Kubernetes security: RBAC with least privilege; namespaces; network policies (bonus); admission policies (bonus).

The relationship to make explicit: security gates must be tied to your versioning and deployment process. A vulnerability report is not “a PDF at the end”; it is a pipeline output that blocks releases until remediated or consciously accepted with justification.

Reliability: scaling, resilience, backups, incident simulation :

DevOps is also about running the system reliably. Your platform must show that it can survive expected failures: pod restarts, temporary database issues, increased traffic, and configuration mistakes. This section is where your monitoring and GitOps practices become operationally meaningful.

Minimum reliability requirements:

- Demonstrate scaling: increase replicas manually or via HPA and show the impact on latency/throughput.
- Demonstrate resilience: kill a pod and show self-healing; simulate a failing dependency and show graceful degradation.
- Backups: implement at least one automated backup of PostgreSQL (and restore) and document the process.
- Runbook: a short “how to operate” document for common tasks (deploy, rollback, rotate secrets, restore DB, check dashboards). A powerful demo is an incident drill: you intentionally break something (wrong config, bad release, expired secret), then use observability to detect it,

Git to trace it, and GitOps rollback to recover. This shows the real relationship between delivery and operations.

Deliverables checklist and grading rubric :

Your submission must be complete and reviewable. The evaluator should be able to clone your repositories, follow documented steps, and reproduce the platform (or at least deploy it in the provided lab).

Mandatory deliverables :
- Source code repositories (application + GitOps).
- CI pipeline definitions (GitLab CI and/or Jenkinsfile).
- Dockerfiles and docker-compose for local development.
- Kubernetes manifests/Helm charts with environment configuration.
- Terraform code (modules + env) and Ansible playbooks/roles.
- ArgoCD configuration (apps/projects) and documented sync strategy.
- Observability stack configured: dashboards + alerts + logging + tracing.
- Security evidence: scan reports, Vault usage, remediation notes.
- Documentation: architecture diagram, README, runbooks, backup/restore procedure.
- Agility Methodologies
- Presentation and Live demonstration for the team.

Live demo script (what you must show) :
1) A code change was merged through MR after pipeline success.
2) New image pushed to registry and versioned (tag).
3) GitOps change triggers ArgoCD sync into dev, then promotion to prod.
4) Observability: dashboards + a trace + logs for one request.
5) Security: at least one scan finding and remediation, plus Vault usage.
6) Reliability: scaling or rollback performed live.
7) The full workflow for the project while presenting it.